

⇒ UNIT-1

ONE - SHOT

BIG DATA

⇒ Topics :

⇒ Big Data

⇒ Types of Big data / different forms of Big Data

⇒ Big Data Architecture

⇒ characteristics or 5V's of Big Data

⇒ Importance of Big - data

⇒ Application of Big - data

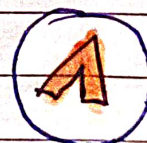
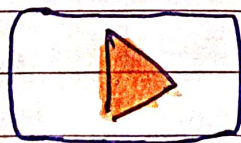
⇒ Three dimensions of Big data

⇒ Analysis VS reporting

⇒ Traditional Reporting & Advanced Analytics

⇒ Traditional Data VS Big Data

⇒ Big Data Analytics



LIKE SHARE & SUBSCRIBE

⇒ Data

- Data is a collection of facts, figures, or details that we can use, study, or analyze.
- It helps us to understand thing, make choices, or solve problems.
- It can be : Number, words, images, sound videos.

⇒ Big Data :-

- Big Data means a huge amount of data - so big that regular computer's can't handle it easily.
- Big data refers to extremely large and complex sets of data that are generated at high speed from various sources.
- To understand it we need advance tools and method.

⇒ Types of Big data / Different Forms of Big Data

1. Structured Data

- Data that is organized and easy to store in rows and columns - like in an Excel sheet or database.
- Eg Bank records, student marks

2. Unstructured Data

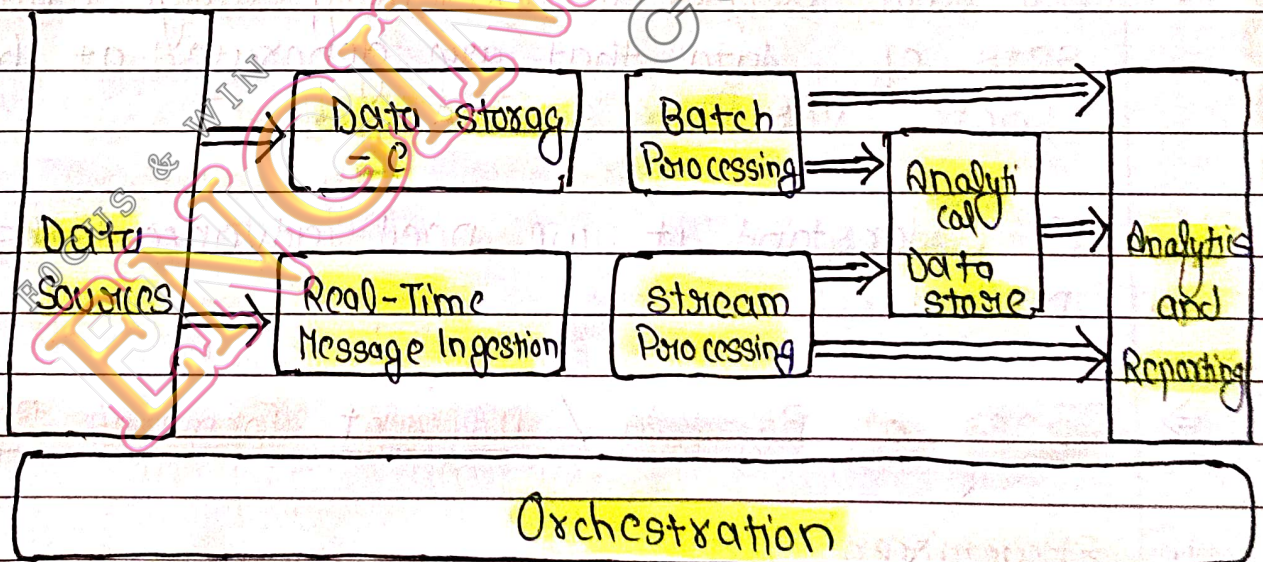
- Data that has no clear format or structure
- Eg Social Media posts, photos, Email

3. Semi-structured Data

- Data that is partially organized - not fully structured, but not fully unstructured either.

Q Elaborate various components of Big data architecture ? AKTU 2021-2022, 2023-2024

Sol Big Data architecture is designed to handle the ingestion, processing, and analysis of data that is too large or complex for traditional database systems



Big Data Architecture

1. Data Sources (where data comes from)

- Data can come from:
 - Social Media (facebook, twitter)
 - websites and apps
 - Sensors, IoT devices
 - Databases

2. Data Ingestion (How data enters the system)

- This step brings data in to the system in real time (fast) or in batches (slow)
- Tools : Apache Kafka, Apache Flume, Sqoop

3. Data Storage (where data is kept)

- Big data needs special storage systems that can handle a lot of different data types
- Tools :
 - Hadoop HDFS
 - Amazon S3
 - NOSQL databases like MongoDB, Cassandra

4. Data Processing (How data is cleaned and used)

- This step cleans, filters, and analyzes the data to make it useful
- Two types :
 - Batch Processing - Handles large data at once.
(Hadoop MapReduce, Apache Spark)
 - Real-time Processing - Handles data as it comes (Apache Storm, Spark Streaming)

5. Data Analysis (Finding Insights)

- Now, the processed data is analyzed to find useful information or patterns
- Tools: Apache Hive, Pig, Spark SQL

6. Data Visualization (Showing results)

- This part turns data into charts, graphs, dashboards to make it easy to understand.
- Tools: Power BI, Tableau, Kibana

7. Data Management & Monitoring

- Make sure the system is secure, runs smoothly, and uses data properly
- Includes: Security, data backup, performance tracking

⇒ Characteristics or 5V's of Big Data:

1. **Volume** - Huge amount of data
2. **Velocity** - Data comes in very fast
3. **Variety** - Different types of data
4. **Veracity** - Accuracy and quality of the data
5. **Value** - Useful insights gained from the data

Importance of Big-data :-

- Handles massive data efficiently
- Helps in better and faster decisions
- Improves customer satisfaction
- Detect fraud or risks
- Boosts Innovation and growth.

Applications of Big-data :-

1. Entertainment & Media.
2. E-commerce
3. Healthcare
4. Banking & Finance
5. Transportation & Logistics
6. Agriculture, smart cities, Education.

Q Detail about the three dimensions of Big data.

Sol 1. Volume (How much data)

→ Volume refers to the amount of data being generated and stored

→ In Big Data, the data is huge - ranging from terabytes (TB) to petabytes (PB) and beyond

Examples :-

→ Facebook generates over 4 petabytes of data per day.

2. Velocity (How Fast Data is Generated?)

→ Velocity is about the speed at which data is generated, collected, and processed

Examples :-

→ Stock market updates every second

3. Variety (What Types of Data?)

→ Variety refers to the different types and sources of data - structured, semi-structured and unstructured

Examples :

→ Structured : Excel sheets, databases

→ Semi-structured : JSON, XML files

→ Unstructured : Images, video, audio, emails

Q
sol

Discuss in detail the different forms of Big data.
Already Done!

Q

Detail about the analysis vs reporting while introducing the Big Data.

Feature

Reporting

Analysis

Purpose

To summarize and present existing data

To interpret data, find patterns, and support decision-making

Focus

Past data

Future strategies and business improvements

Approach

Descriptive (what happened)

Diagnostic, predictive and prescriptive

Tools used

Excel, Power BI, Tableau

Python, R, M.L tools

Output charts, tables, dashboards Models, predictions, recommendations

User Level Business users, managers Data analysts, data scientists

Time Orientation Mostly past and current data Past, present and future

Q How does advanced analytics go beyond traditional reporting to uncover hidden patterns, trends, and correlations in data.

Sol Traditional Reporting?

- Traditional reporting is like looking in the rear-view mirror.
- It tells you what happened in the past using :
 - Tables
 - Charts
 - Graphs

Advanced Analytics?

- Advanced analytics is like a smart detective
- It goes deeper to find reasons, patterns, and future trends using techniques like
 - Statistics
 - Machine learning
 - Predictive modeling

Q. What do each of these terms represent in the context of Big Data, and why are they essential considerations for data management and analysis?

Sol

1. **Volume** — "How much data?"

Why it matters: Traditional tools can't handle this massive scale. We need scalable storage (like Hadoop, Cloud Storage) and fast processing tools.

2. **Velocity** — "How fast is data coming in?"

Why it matters: For real-time decisions (like fraud detection, traffic control), we need systems that can analyze data instantly.

3. **Variety** — "What types of data?"

Why it matters: Tools must be flexible to handle and integrate different data types effectively.

4. **Veracity** — "How accurate and trustworthy is the data?"

Why it matters: Poor-quality or inconsistent data can lead to wrong analysis and bad decisions.

5. **Value** — "What useful insights can we get from the data?"

Why it matters: Simply collecting large amount of data is useless, unless we can turn it into valuable insights.

Date : ___ / ___ / ___

Q Traditional Data Vs Big Data

Parameters	Traditional Data	Big Data
Volume	GB	TB or PB
Generate Rate	Per hr, Per Day	Every sec or minute
Structure	Structured	Semistructured or unstructured
Data Source	Centralized	Fully Distributed
Data integration	Easy	Difficult
Data Store	RDBMS	HDFS, NoSQL
Access	Interactive	Batch or near real time
Data Structure	Static Schema	Dynamic Schema
Scaling Potential	Non-linear	Somewhat close to linear

Q

Big Data Analytics ?

Big data analytics is the use of advanced analytic technique against very large, diverse data sets that include structured, semi-structured and unstructured data, from different sources, and in different sizes from terabytes to zettabytes

Types :

1. Descriptive Analysis

What is happening now based on incoming data

2. Predictive Analysis

What might happen in future

3. Prescriptive Analysis

What action should be taken

4. Diagnostic Analysis

Why/what did it happen.

LIKE SHARE & SUBSCRIBE

UNIT-1 COMPLETE

THANK YOU FOR WATCHING

⇒ UNIT- 2

BIG DATA

ONE - SHOT

⇒ Topics :

- ⇒ History of Hadoop
- ⇒ Apache Hadoop
- ⇒ Hadoop Distributed File System
- ⇒ Components of Hadoop
- ⇒ Hadoop Ecosystem
- ⇒ MAP Reduce Function
- ⇒ YARN Implementation

LIKE SHARE & SUBSCRIBE



~

⇒ Hadoop

- Hadoop is a free software tool made by the Apache Software Foundation.
- It helps store and process a lot of data using many computer working together
- Hadoop is great for companies that need to handle big data, like social media platform, search engines or online stores

⇒ History of Hadoop

- 2003: Google shared a paper about GFS - a way to store big data across many computers.
- 2004: Google introduced Map Reduce - a method to process big data using multiple computer.
- 2005: Doug Cutting and Mike Cafarella started building Hadoop based on Google's ideas, while working on a project called Nutch
- 2006: Hadoop became its own project under Apache, and Yahoo! began supporting it
- 2008: Yahoo built the project, largest Hadoop system then, and Hadoop became an official Apache top-level project

Date : ___ / ___ / ___

- 2011 : Hadoop 1.0 was released with HDFS (storage) and MapReduce (processing)
- 2013 : Hadoop 2.0 came out with YARN, which helped manages resources better.
- 2017 - Now : Hadoop grew with tools like Hive, Pig and Spark, but some companies now use newer or cloud based system.

Apache Hadoop

Apache Hadoop is a free, open-source software framework that helps store and process large amount of data using many computers working together.

It's useful for handling Big data that is too large or complex for traditional systems

Main Parts of Hadoop

1. HDFS (Hadoop Distributed File System) :-
Stores big data by splitting it into blocks and spreading it across multiple computers
2. MapReduce :-
A programming model that processes data in parallel (many task run at the same time)
3. YARN (Yet Another Resource Negotiator) :-

Date : ___ / ___ / ___

Manages and allocates system resources to different applications

4. Hadoop Common :
Shared tools and libraries used by all the Hadoop components

⇒ Hadoop Works ?

- Data is initially divided into uniform sized blocks of 128 MB or 64 MB.
- These files are distributed across various clusters for processing.
- HDFS Supervised the overall processing.
- Blocks are replicated for handling hardware failure.
- Performing the task that place between MapReduce and further stages.
- Send processed data to the certain computers.
- Writes the debugging logs for each jobs assign.

Advantage

- Ability to store a large amount of data.
- Distributed computing
- Cost effective
- Parallel processing

Disadvantage

- Not very effective for small data
- Security concerns
- Has stability issue

Q

Discuss Hadoop Distributed File system (HDFS) and how does it works

Sol

The Hadoop Distributed File system is a core component of the Apache Hadoop ecosystem.

- It is a scalable, fault-tolerant, distributed file system design to run on commodity hardware and to manage large volume of data efficiently.

⇒

Key Feature of HDFS

- Scalability
- Fault tolerance
- High throughput
- Write once, Read many
- Commodity Hardware

⇒

HDFS Architecture

HDFS follows a master / slave architecture, comprising two main components

1. NameNode (Master)

- Manages the file system namespace.
- Maintains the metadata
- Does not store the actual data

2. DataNode (slaves)

- Store the actual data in blocks.
- Perform block creation, deletion and replication upon instruction from the NameNode.

- Periodically report back to the NameNode with a heartbeat and block report

⇒ HDFS works :-

1. File Storage (write operation)

- Files are split into fixed-size blocks (default 128/256 MB)
- Blocks are replicated (default 3 copies) and distributed across DataNodes
- The client contacts the NameNode for block placement
- Data is written directly to DataNodes in a pipeline
- The NameNode stores Metadata about blocks locations

2. File Retrieval (Read operation)

- The client requests file metadata from the NameNode
- The NameNode returns block locations and replica info.
- The client reads in parallel from the nearest DataNodes for efficiency

3. Replication & Fault Tolerance

- NameNode monitors DataNodes via heartbeats
- On failure, it triggers block replication to maintain redundancy and availability

Q Explain Hadoop ecosystem in detail.

Sol The Hadoop Ecosystem is a comprehensive suite of tools and technologies built around the Hadoop framework to handle large-scale data processing in a distributed computing environment.

⇒ It includes modules for data storage, processing, ingestion, analysis, workflow management and coordination.

Core component of the Hadoop Ecosystem

1. Hadoop Common
2. Hadoop Distributed File System (HDFS)
 - ↳ used to store large datasets across clusters.
3. Hadoop YARN (Yet another Resource Negotiator)
 - ↳ resource manager of the ecosystem.
 - ↳ Manages computing resources and job scheduling across the cluster.
4. MapReduce
 - ↳ A programming model for batch data processing.
 - ↳ Splits jobs into Map and reduce tasks.

⇒ Apache Hive

- Data warehouse infrastructure built on Hadoop
- Provides an SQL-like language called HiveQL
- Converts HiveQL queries into MapReduce, Tez or Spark jobs

⇒ Apache Pig

A high-level platform for creating MapReduce programs using a scripting language called Pig Latin.

Suited for data transformation and ETL tasks

⇒ Apache HBase

A NoSQL database built on HDFS.

Provides real-time read/write access to large datasets.

⇒ Apache Spark

An in memory data processing engine that can run standalone or on top of YARN.

Faster than MapReduce

⇒ Apache Flume

A distributed system for efficiently collecting, aggregating and moving large amount of log data to HDFS or HBase.

⇒ Apache Sqoop

Used for transferring data between relational databases and HDFS.

SUBSCRIBE: MULTIACTIONS

Date : ___ / ___ / ___

Q Discuss the detailed architecture of YARN along with its components.

Sol YARN is the resource management and job scheduling layer of the Hadoop ecosystem, introduced in Hadoop 2.x to improve scalability, cluster utilization, and to support multiple processing engines.

⇒ YARN follows a master-slave architecture and includes the following key components:

1. **Resource Manager (RM) - Global Master**
 - Runs on the master node
 - Manages the global resource allocation in the cluster
 - Two main modules:
 - **Scheduler**: Allocates resources based on policies (FIFO, capacity, fair)
 - **Application Manager**: Manages job submission, monitors app lifecycles
2. **Node Manager (NM) - Per-Node Agent**
 - Runs on each worker node
 - Reports resource usage (CPU, Memory) to the RM
 - Launches and monitors containers on the node as directed by the RM
 - Manages logs, resource isolation and health monitoring

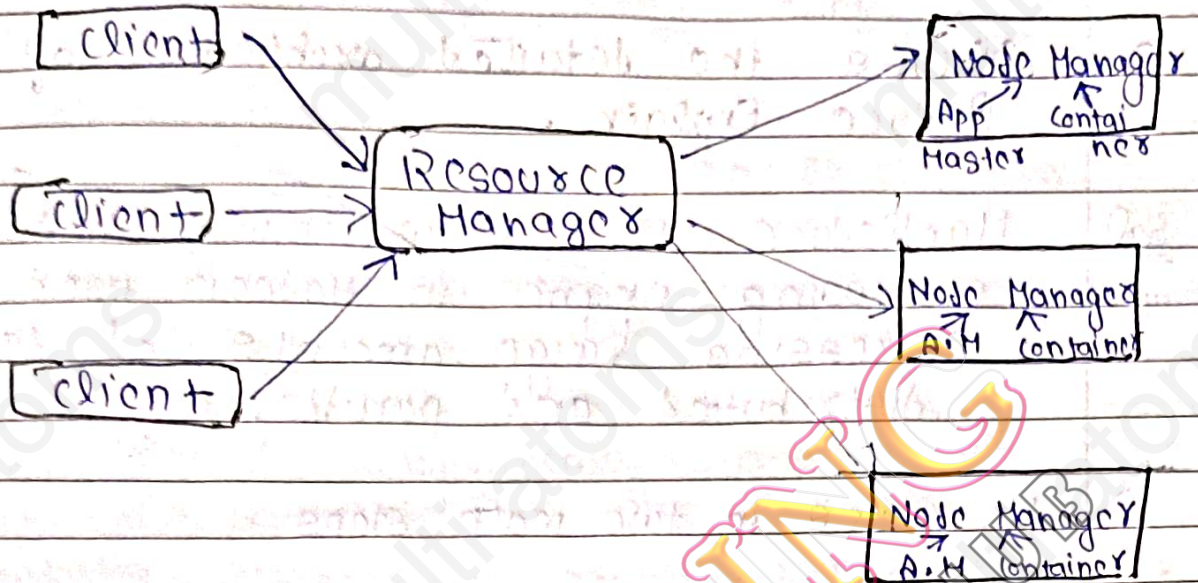
3. Application Master (AM) - Per-Application Manager
- Created per application (job) by the Resource Manager
 - Manages the application lifecycle :
 - Requests resources from the RM
 - Coordinates execution within allocated container
 - Handles failures and restarts of tasks
 - Terminates when the application finishes

4. Container - Resource Unit
- The basic execution unit in YARN
 - Encapsulates resources on a single node
 - Allocated by RM and launched by NM
 - Used by AM to run individual tasks

⇒ Working of YARN :-

1. Client submits job to Resource Manager
2. Resource Manager allocates a container to launch the Application Master
3. Application Master registers with the RM and requests containers for tasks
4. Resource Manager grants containers, and tasks are launched by respective Node Managers
5. Tasks run in containers, reporting status to the Application Master
6. Upon job completion, the AM de-registers, and the containers are released.

Date : ___ / ___ / ___



Advantages :

- 1] Flexibility
- 2] Resource Management
- 3] Scalability
- 4] Improved Performance
- 5] Security

Subscribe MultiAtoms

&
MultiAtoms Plus

Q Discuss the detailed architecture of Map-Reduce Explain.

Ans MapReduce is a programming model and processing engine in Hadoop used for processing large amounts of data in distributed and parallel way.

⇒ It works in two main steps

- Map → Breaks down tasks and processes them in parallel.
- Reduce → Gathers and summarizes the results.

⇒ MapReduce Architecture Components

1. Client (User Program)

- Submit the map reduce job to Hadoop.
- Job contains : Map function, Reduce function and input data path.

2. JobTracker (in Hadoop 1.x) or YARN Resource Manager + ApplicationMaster (in Hadoop 2.x)

- Accepts the job from the client.
- Breaks it into smaller tasks.
- Manages Job scheduling and resource allocation.

Date : ___ / ___ / ___

3. Map Task

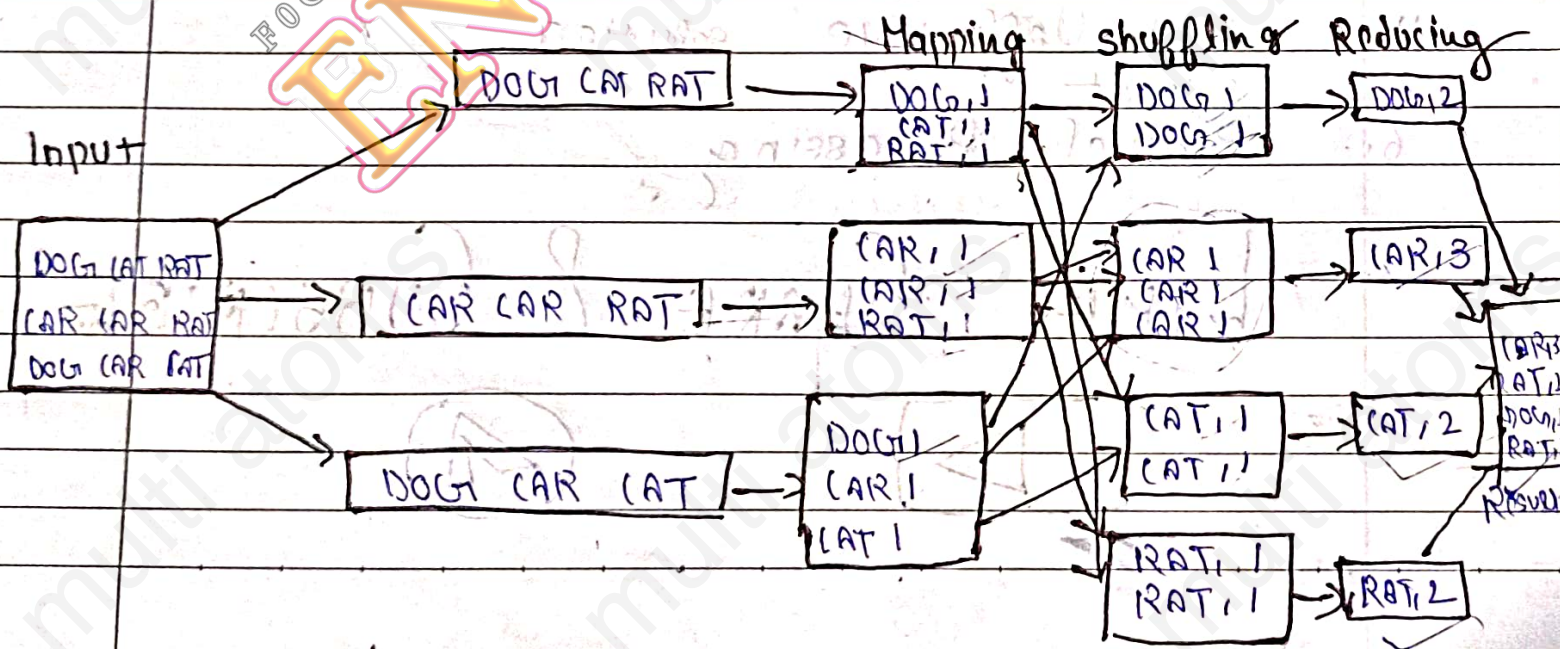
- Reads the input data split (usually from HDFS)
- Processes the data and converts it into key-value pairs
- Intermediate results are written to local disk

4. Shuffle and Sort

- The system automatically groups all values with the same key together
- Transfer intermediate data from mapper to reducers
- Sorts keys so that all values for a key go to the same reducer

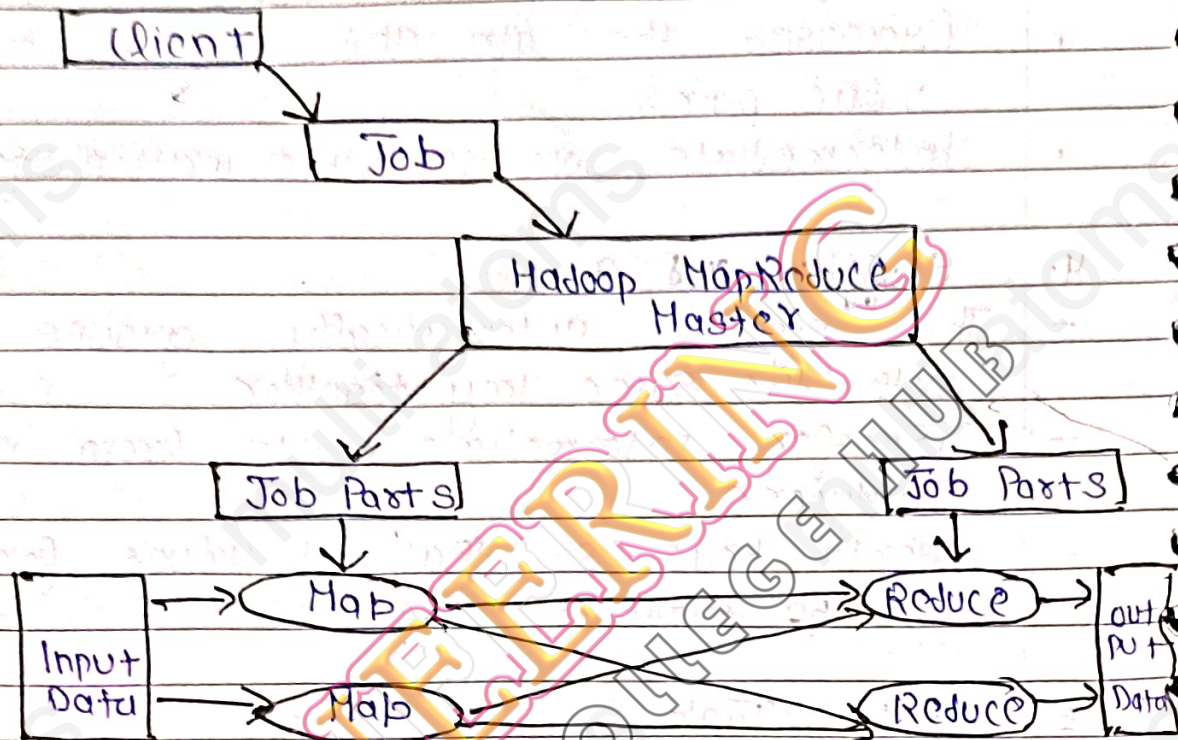
5. Reduce Task

- Takes grouped key-value pairs.
- Applies the reduce function to combine values
- Outputs the final result to HDFS



Date : ___ / ___ / ___

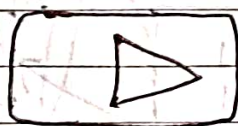
Map Reduce Architecture



Map Reduce Features

1. Scalability
2. Flexibility
3. Security and Authentication
4. Cost-effective solution
5. Fast
6. Parallel Processing

LIKE SHARE & SUBSCRIBE



BIG DATA ONE SHOT UNIT-3

III	HDFS (Hadoop Distributed File System): Design of HDFS, HDFS concepts, benefits and challenges, file sizes, block sizes and block abstraction in HDFS, data replication, how does HDFS store, read, and write files, Java interfaces to HDFS, command line interface, Hadoop file system interfaces, data flow, data ingest with Flume and Scoop, Hadoop archives, Hadoop I/O: compression, serialization, Avro and file-based data structures. Hadoop Environment: Setting up a Hadoop cluster, cluster specification, cluster setup and installation, Hadoop configuration, security in Hadoop, administering Hadoop, HDFS monitoring & maintenance, Hadoop benchmarks, Hadoop in the cloud
-----	--

Q1. Explain the core concepts of HDFS, including NameNode, DataNode, and the file system namespace ?

HDFS (Hadoop Distributed File System) is like a giant storage system that splits big files into smaller pieces and spreads them across many computers. Here's how it works in simple terms:

1. File System Namespace (The Big Index)

- Think of it like a **table of contents** for all your files.
- It keeps track of:
 - File & folder names
 - Who can access them
 - Where each piece (block) of the file is stored

2. NameNode (The Boss)

- The **main manager** that knows everything about the files.
- Stores **only the metadata** (file names, permissions, block locations).
- **Does NOT store actual data**—just the info about where it is.
- If the NameNode crashes, the whole system stops (so it's very important!).

3. DataNode (The Workers)

- These are the **actual storage machines** that hold the file pieces (blocks).
- They constantly report back to the NameNode saying, *"I'm alive and here's what I have!"*
- If a DataNode fails, HDFS makes **copies (replicas)** of the data on other machines.

Key Features

- **Big Files Only** – Best for large files (like TBs of data), not small ones.
- **Fault-Tolerant** – If one machine dies, your data is safe because of copies.
- **Fast Batch Processing** – Good for analytics (like reading huge files at once), not for quick edits.

Q2 Write the benefits and challenges of HDFS.?

Benefits of HDFS

1. Handles Massive Data (Scalability)

- Stores **petabytes (PBs)** of data across thousands of machines.
- Easily scales by adding more **DataNodes** (storage servers).

2. Fault-Tolerant (No Data Loss)

- **Automatically creates multiple copies (replicas)** of each data block (default = 3 copies).
- If one **DataNode** fails, data is still available from other nodes.

3. Cost-Effective Storage

- Runs on **cheap commodity hardware** (regular servers, no need for expensive systems).

4. Optimized for Big Data Processing

- Designed for **batch processing** (reading/writing large files sequentially).
- Works great with **MapReduce, Spark, and other big data tools**.

5. Data Locality (Faster Processing)

- Moves computation to **where data is stored** instead of moving data to computation.
- Reduces network traffic and speeds up analytics.

✗ Challenges of HDFS

1. Not Good for Small Files

- Designed for **large files (GBs/TBs)**.
- Storing too many small files **overloads the NameNode** (since it keeps metadata in memory).

2. High Latency (Not Real-Time)

2. High Latency (NOT Real-Time)

- Built for **batch processing**, not fast queries.
- Not suitable for **real-time analytics** (like databases).

3. Single Point of Failure (NameNode Risk)

- If the **NameNode** crashes, the whole system becomes unavailable.
- Solutions like **HDFS High Availability (HA)** help, but add complexity.

4. Limited Write Flexibility

- Follows "Write Once, Read Many" (WORM) model.
- Files **cannot be modified** after writing (only appended or rewritten).

5. High Storage Overhead (Due to Replication)

- Default **3x replication** means storing **3 copies** of everything.
- Increases storage costs but ensures fault tolerance.

Q3. Explain how HDFS stores, reads, and writes files. Describe the sequence of operations involved in storing a file in HDFS, retrieving data from HDFS, and writing data to HDFS. ?

HDFS follows a structured approach for storing, reading, and writing files across a distributed cluster. Below is a step-by-step breakdown of each process.

1. Storing a File in HDFS (Write Operation)

Step-by-Step Process:

1. Client Initiates Write Request

- The client (user or application) requests to write a file to HDFS.
- The file is split into **fixed-size blocks** (default: 128MB or 256MB).

2. NameNode Assigns DataNodes

- The client contacts the **NameNode**, which checks permissions and file existence.
- The NameNode selects **3 DataNodes** (default replication factor) for each block and returns their addresses.

3. Pipeline Creation & Data Transfer

- The client writes the first block to the **first DataNode**.
- The first DataNode forwards the block to the **second DataNode**, which forwards it to the **third DataNode** (forming a **pipeline**).

- Each DataNode stores the block and sends an **acknowledgment (ACK)** back.

4. Repeat for All Blocks

- The process repeats for all blocks of the file.

5. NameNode Updates Metadata

- Once all blocks are stored, the NameNode updates the **metadata** (file name, block locations, permissions).

2. Reading a File from HDFS (Read Operation)

Step-by-Step Process:

1. Client Requests Read

- The client asks the **NameNode** for the file's block locations.

2. NameNode Returns Block Metadata

- The NameNode checks permissions and returns:
 - **List of blocks** making up the file.
 - **Locations (DataNodes)** of each block (sorted by network proximity).

3. Client Reads Directly from DataNodes

- The client reads blocks **in parallel** from the closest DataNodes.
- If a DataNode fails, the client automatically switches to a **replica**.

4. Blocks Reassembled into Original File

- The client combines the blocks in order to **reconstruct** the file.

3. Writing Data to an Existing File (Append Operation)

HDFS primarily follows a "Write Once, Read Many" (WORM) model, but limited appends are possible.

Step-by-Step Process:

1. Client Requests Append

- The client asks the NameNode to append data to an existing file.

2. NameNode Checks Conditions

- Ensures the file exists and supports appends.
- Locates the last block of the file (if incomplete, it is filled first).

3. New Data is Written

- The client writes new data to the last block (if space remains).
- If the block is full, a **new block** is allocated and replicated.

4. Metadata Updated

- The NameNode updates the file's metadata to reflect changes.

- The NameNode updates the file's metadata to reflect changes.

Q4 Describe the considerations for deploying Hadoop in a cloud environment. What are the advantages and challenges of running Hadoop clusters on cloud platforms like Amazon Web Services (AWS), Microsoft Azure, and Google Cloud Platform (GCP).?

Deploying **Hadoop in a cloud environment** like **AWS, Azure, or GCP** offers flexibility and scalability—but also comes with important considerations. Below is a breakdown of **key factors**, along with **advantages** and **challenges** of running Hadoop in the cloud.

✓ Advantages

1. **Elastic & Scalable** – Auto-scaling clusters, pay-as-you-go pricing.
2. **Lower Maintenance** – Managed services (AWS EMR, Azure HDInsight, GCP Dataproc).
3. **Cost-Efficient** – Spot/preemptible instances for batch jobs.
4. **Durable Storage** – Cloud-native storage (S3, GCS) > HDFS.
5. **Built-in HA/DR** – Multi-region replication.

✗ Challenges

1. **Network Latency** – Slow reads if compute/storage are separated.
2. **Security Risks** – Shared responsibility model (user manages Hadoop security).
3. **Variable Performance** – Noisy neighbors, network bottlenecks.
4. **Hidden Costs** – Egress fees, idle clusters, over-provisioning.
5. **Vendor Lock-in** – Hard to migrate from cloud-native services.

Q5. Discuss in brief about the cluster specification. Describe how to setting up a Hadoop Cluster?

A Hadoop cluster is a group of computers (nodes) working together to store and process huge amounts of data. It has:

- **1 Master Node (Manager):** Controls everything (NameNode + ResourceManager).
- **Many Worker Nodes (Workers):** Store data and do computations (DataNodes).

How to Set Up a Hadoop Cluster?

Step 1: Get the Machines Ready

- **Master Node:** Needs good CPU & RAM (e.g., 8 cores, 32GB RAM).
- **Worker Nodes:** Need lots of storage (e.g., 16 cores, 64GB RAM, 10TB HDD each).

- **All Nodes:** Install Java (JDK 8/11) and SSH (for remote access).

Step 2: Install & Configure Hadoop

1. **Download Hadoop** and extract it on all machines.
2. **Edit Config Files** (tell Hadoop how to work):
 - `core-site.xml` → Set master's address.
 - `hdfs-site.xml` → Set data copy count (default = 3).
 - `yarn-site.xml` → Configure processing (YARN).

Step 3: Start the Cluster

1. **Format the Master** (like formatting a hard drive):

```
bash

hdfs namenode -format
```

2. **Start Hadoop Services:**

```
bash

start-dfs.sh      # Starts storage (HDFS)
start-yarn.sh     # Starts processing (YARN)
```

Step 4: Check if It Works

- **View Live Nodes:**

```
bash

hdfs dfsadmin -report
```

- **Web Dashboard:** Open browser and go to:
 - `http://<master-ip>:9870` (HDFS status).
 - `http://<master-ip>:8088` (YARN jobs).

Q. Demonstrate the design of HDFS and concept in detail. ?

Q. Examine how a client read and write data in HDFS. ?

BIG DATA ONE SHOT UNIT-4

IV

Hadoop Eco System and YARN: Hadoop ecosystem components, schedulers, fair and capacity, Hadoop 2.0 New Features - NameNode high availability, HDFS federation, MRv2, YARN, Running MRv1 in YARN.

NoSQL Databases: Introduction to NoSQL

MongoDB: Introduction, data types, creating, updating and deleting documents, querying, introduction to indexing, capped collections

Spark: Installing spark, spark applications, jobs, stages and tasks, Resilient Distributed Databases, anatomy of a Spark job run, Spark on YARN

SCALA: Introduction, classes and objects, basic types and operators, built-in control structures, functions and closures, inheritance.

Q1 State features of Apache Spark and also explain three ways of how Spark can be built with Hadoop components.?

Features of Apache Spark (Simple Explanation)

Apache Spark is a fast and flexible big data processing engine with these key features:

1. **Speed** – Runs up to **100x faster** than Hadoop MapReduce by storing data in memory (RAM) instead of disk.
2. **Easy to Use** – Supports **Python, Java, Scala, and R**, making it accessible to many developers.
3. **Real-Time Processing** – Can handle **live data streams** (like tweets or sensor data) in real time.
4. **Works Everywhere** – Can run on **Hadoop, Kubernetes, cloud (AWS, Azure), or standalone clusters**.
5. **Smart Optimization** – Waits to execute tasks until necessary (**lazy evaluation**) to save time.
6. **Recovers from Failures** – If a machine crashes, Spark can **rebuild lost data** automatically.
7. **Supports SQL, ML, and Graphs** – Can run **SQL queries, machine learning, and graph processing** in the same system.

3 Ways Spark Works with Hadoop (Simple Explanation)

Spark can be used with Hadoop in different ways depending on what parts of Hadoop you need:

1. Spark Alone (But Using HDFS for Storage)

- **How it works:**
 - Spark runs **by itself** (no Hadoop YARN or MapReduce).
 - But it still uses **HDFS (Hadoop's storage system)** to read/write data.
- **When to use:**
 - You want **fast Spark processing** but still need Hadoop's reliable storage.

2. Spark on YARN (Sharing Resources with Hadoop)

- **How it works:**

- Spark runs **on top of YARN** (Hadoop's resource manager).
- Shares the same cluster with **MapReduce, Hive, etc.**
- **When to use:**
 - You have a **Hadoop cluster** and want Spark to **share resources** (CPU, memory) with other Hadoop jobs.

3. Spark Replacing MapReduce (Faster Processing with Hadoop Tools)

- **How it works:**
 - Instead of using **slow MapReduce**, you use **Spark for processing**.
 - Still keeps **HDFS (storage)**, **YARN (resource manager)**, and tools like **Hive & HBase**.
- **When to use:**
 - You want **Spark's speed** but still need Hadoop's ecosystem (like Hive for SQL queries).

Q2 Explain how CRUD operations with example are performed in MongoDB. ?

CRUD Operations in MongoDB (with Examples)

MongoDB is a **NoSQL database** that stores data in **JSON-like documents** (called **BSON**). CRUD stands for **Create, Read, Update, Delete**—the four basic operations in any database.

1. Create (Insert) Documents

Add new documents into a collection.

Methods:

- `insertOne()` → Adds a single document.
- `insertMany()` → Adds multiple documents.

Examples:

Insert a Single Document

javascript

```
db.users.insertOne({
  name: "Alice",
  age: 25,
  email: "alice@example.com"
});
```

Output:

json

```
{
  "acknowledged": true,
  "insertedId": ObjectId("5f8d...")
}
```

Insert Multiple Documents

javascript

```
db.users.insertMany([
  { name: "Bob", age: 30, email: "bob@example.com" },
  { name: "Charlie", age: 35, email: "charlie@example.com" }
]);
```

Output:

json

```
{
  "acknowledged": true,
  "insertedIds": [
    ObjectId("5f8d..."),
    ObjectId("5f8d...")
  ]
}
```

2. Read (Query) Documents

Retrieve documents from a collection.

Methods:

- `find()` → Returns all matching documents.
- `findOne()` → Returns the first matching document.

Examples:

Find All Documents

javascript

```
db.users.find();
```

Output:

json


```
[
  { "_id": ObjectId("..."), "name": "Alice", "age": 25, "email": "alice@example.com" },
  { "_id": ObjectId("..."), "name": "Bob", "age": 30, "email": "bob@example.com" }
]
```

Find with Filter (Condition)

javascript

```
db.users.find({ age: { $gt: 25 } }); // Users older than 25
```

Output:

json

```
[
  { "_id": ObjectId("..."), "name": "Bob", "age": 30, "email": "bob@example.com" }
]
```

Find One Document

javascript

```
db.users.findOne({ name: "Alice" });
```

Output:

json

```
{ "_id": ObjectId("..."), "name": "Alice", "age": 25, "email": "alice@example.com" }
```

3. Update Documents

Modify existing documents.

Methods:

- `updateOne()` → Updates the first matching document.
- `updateMany()` → Updates all matching documents.
- `replaceOne()` → Replaces the entire document.

Examples:

Update One Document

javascript

```
db.users.updateOne(
  { name: "Alice" }, // Filter
```



```
{ $set: { age: 26 } } // Update operation
);
```

Output:

json

```
{
  "acknowledged": true,
  "matchedCount": 1,
  "modifiedCount": 1
}
```

Update Many Documents

javascript

```
db.users.updateMany(
  { age: { $lt: 30 } }, // Filter (age less than 30)
  { $set: { status: "young" } } // Update
);
```

Output:

json

```
{
  "acknowledged": true,
  "matchedCount": 2,
  "modifiedCount": 2
}
```

4. Delete Documents

Remove documents from a collection.

Methods:

- `deleteOne()` → Deletes the first matching document.
- `deleteMany()` → Deletes all matching documents.

Examples:

Delete One Document

javascript

```
db.users.deleteOne({ name: "Alice" });
```

Output:

json

```
{
  "acknowledged": true,
  "deletedCount": 1
}
```

Delete Many Documents

javascript

```
db.users.deleteMany({ age: { $gt: 30 } }); // Delete users older than 30
```

Output:

json

```
{
  "acknowledged": true,
  "deletedCount": 1
}
```

Q1 . With the help of suitable example, explain how CRUD operations are performed in MongoDB.

Q2 . Explain the operations for creating, updating, and deleting documents in MongoDB. What are the MongoDB CRUD operations, and how are they used to manipulate data in collections?

Q3 Summarize the role of indexing in MongoDB using an example?

What is Indexing?

Indexing in MongoDB is like a **book's index**—it helps the database **find data faster** without scanning every document (similar to how an index helps you quickly locate a topic in a book instead of reading every page).

Why Use Indexes?

- Improves query speed (especially for large collections).
- Reduces database workload (avoids full collection scans).

• Speeds up sorting and filtering

- Speeds up sorting and filtering.

Example: Without vs. With Indexing

1. Collection Without Index (Slow Search)

Imagine a `users` collection with **1 million documents**:

```
javascript

db.users.insertMany([
  { name: "Alice", age: 25, city: "New York" },
  { name: "Bob", age: 30, city: "London" },
  // ... 999,998 more documents
]);
```

Query Without Index (Full Scan)

```
javascript

db.users.find({ city: "London" }); // MongoDB scans ALL 1M documents!
```

- **Problem:** Slow performance (like searching a book page by page).

2. Adding an Index (Fast Search)

Create an index on the `city` field:

```
javascript

db.users.createIndex({ city: 1 }); // 1 = ascending order
```

Query With Index (Fast Lookup)

```
javascript

db.users.find({ city: "London" }); // Uses index → instantly finds matching docs!
```

- **How it works:**
 - MongoDB checks the **index** (like a sorted list of cities).
 - Directly jumps to `"London"` entries **without scanning all documents**.

NoSQL databases are designed for flexibility, scalability, and high performance with unstructured or semi-structured data. They are classified into four main types, each optimized for different use cases.

NoSQL Database Types (Compact Overview)

1. Key-Value Stores

- **Structure:** { key: value } (e.g., user123: { name: "Alice" })
- **Best for:** Caching, sessions, fast lookups
- **Examples:** Redis, DynamoDB

2. Document Stores

- **Structure:** JSON/BSON documents (schema-flexible)
- **Best for:** CMS, user profiles, catalogs
- **Examples:** MongoDB, CouchDB

3. Column-Family Stores

- **Structure:** Columns grouped by row keys (optimized for scaling)
- **Best for:** Time-series, big data analytics
- **Examples:** Cassandra, HBase

4. Graph Databases

- **Structure:** Nodes (entities) + edges (relationships)
- **Best for:** Social networks, fraud detection
- **Examples:** Neo4j, Amazon Neptune

When to Use NoSQL?

- ✓ Flexible schemas (e.g., evolving data)
- ✓ High scalability (distributed systems)
- ✓ Fast reads/writes (e.g., real-time apps)

When to Avoid?

- ✗ Complex transactions (use SQL instead)
- ✗ Strict data integrity (ACID compliance needed)

Difference Between Java and Scala

Feature	Java	Scala
Type System	Statically typed, verbose	Statically typed, with type inference
Syntax	Verbose (requires more boilerplate)	Concise (less boilerplate, more expressive)
Paradigm	Primarily object-oriented (OOP)	Hybrid (OOP + Functional Programming)
Null Handling	Uses <code>null</code> (risks <code>NullPointerException</code>)	Uses <code>Option[T]</code> (safer null handling)
Concurrency	Thread-based (<code>synchronized</code> , <code>Executor</code>)	Actor model (Akka), Futures, immutable data
Lambdas	Introduced in Java 8	Built-in from the start
Immutability	Manual (<code>final</code> keyword)	Encouraged (<code>val</code> , immutable collections)
Interoperability	Runs on JVM	Fully interoperable with Java
Code Example	<pre>List<String> list = new ArrayList<>(); val list = List("Scala", "Java")</pre>	

Key Features of Scala

1. Concise & Expressive Syntax

- Less boilerplate than Java.
- Example:

```
scala  
val doubled = List(1, 2, 3).map(_ * 2) // [2, 4, 6]
```

2. Functional + OOP Hybrid

- Supports **immutable data structures**, **higher-order functions**, and **pattern matching**.
- Example:

```
scala  
val sum = (1 to 10).filter(_ % 2 == 0).sum // Sum of evens
```

3. Type Inference

- Compiler deduces types automatically.
- Example:

scala

```
val name = "Scala" // Compiler infers `String`
```

4. Immutability by Default

- **val** (immutable) vs. **var** (mutable).
- Example:

scala

```
val x = 5 // Immutable  
var y = 10 // Mutable (avoid when possible)
```

5. Pattern Matching

- Like **switch** on steroids.
- Example:

scala

```
def matchTest(x: Int): String = x match {  
  case 1 => "One"  
  case _ => "Other"  
}
```

6. Advanced Concurrency (Akka, Futures)

- **Actors model** (Akka) for distributed systems.
- **Futures & Promises** for async programming.
- Example:

scala

```
import scala.concurrent.Future  
val futureResult = Future { computeSomething() }
```

7. Seamless Java Interop

- Can use Java libraries directly.
- Example:

scala

```
import java.util.ArrayList  
val javaList = new ArrayList[String]()
```

8. Case Classes & Algebraic Data Types (ADTs)

- Simplifies data modeling.
- Example:

- example.

scala

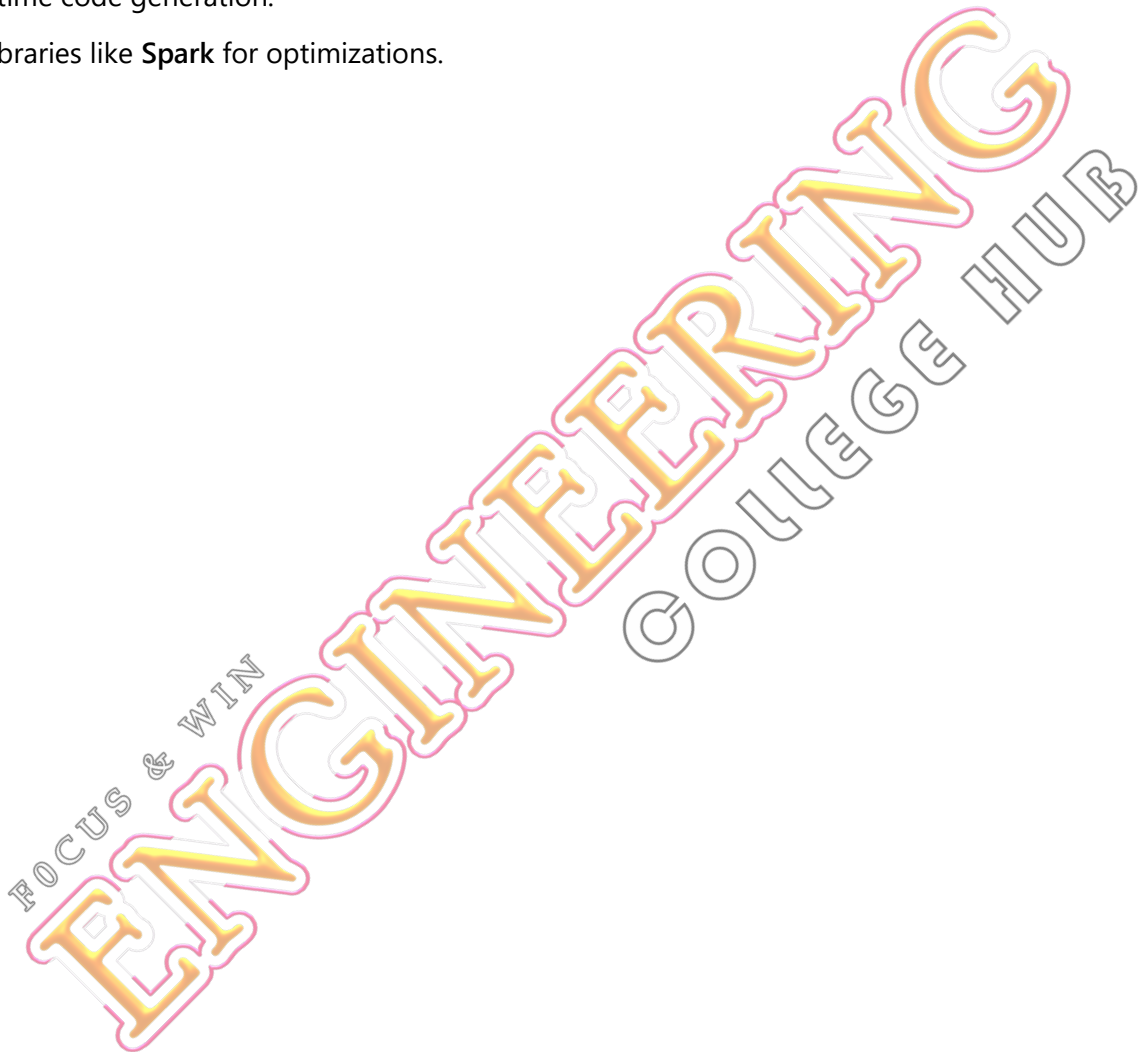
```
case class Person(name: String, age: Int)
val alice = Person("Alice", 25) // No `new` needed
```

9. DSL (Domain-Specific Language) Support

- Enables writing expressive APIs.
- Example (Akka HTTP):

10. Macros & Metaprogramming

- Compile-time code generation.
- Used in libraries like **Spark** for optimizations.



BIG DATA ONE SHOT UNIT-5

V

Hadoop Eco System Frameworks: Applications on Big Data using Pig, Hive and HBase
Pig - Introduction to PIG, Execution Modes of Pig, Comparison of Pig with Databases, Grunt, Pig Latin, User Defined Functions, Data Processing operators,

09

Q1 Explain the architecture of HIVE. Also explain data flow in HIVE.?

Hive is a tool that lets you use **SQL-like queries** to process **big data** stored in **Hadoop**. It makes working with large datasets easy, even if you don't know complex programming.

Hive Clients (How we connect to Hive)

Hive lets you connect and run queries from different programming languages (like Java, Python, C++, etc.) using special drivers.

3 Main Types of Hive Clients:

1. Thrift Client

- Uses **Apache Thrift** to send requests to Hive.
- Works across many programming languages.

2. JDBC Client

- Java programs use **JDBC (Java Database Connectivity)** to connect to Hive through Thrift

3. ODBC Client

- Works like JDBC but is used by other tools (like Excel or BI tools).
- Connects to Hive via HiveServer using Thrift.

Hive Services (Core Components)

1. Beeline

- A command-line tool to run Hive queries.
- Uses JDBC to connect to HiveServer2.
- It is lightweight and user-friendly.

2. HiveServer2

- Handles multiple clients at the same time.
- Lets users run queries and get results.
- Supports JDBC and ODBC clients.

Note:

Old version **HiveServer1** (aka Thrift Server) could not handle multiple users well. That's why **HiveServer2** replaced it.

Hive Internals (How Hive Processes a Query)

1. Hive Driver

- Receives the user's query (written in HiveQL).
- Starts the process and manages the session.

2. Hive Compiler

- Breaks the query into steps.
- Checks for errors using metadata from **Metastore**.
- Creates a **DAG (Directed Acyclic Graph)** — a plan of steps to follow (like map-reduce jobs, data operations, etc.).

3. Optimizer

- Makes the query run faster by rearranging or splitting tasks.

4. Execution Engine

- Runs the final plan using **MapReduce**, **Tez**, or **Spark**.
 - Gets the data from **HDFS** and processes it.
-

Metastore (The Metadata Store)

- Stores info about tables, columns, partitions, and file locations.
- Tells Hive where data is and how it's structured.
- Usually backed by a database like MySQL.

Modes of Metastore:

- **Remote Mode:** Metastore is accessed via network (for non-Java clients).
 - **Embedded Mode:** Metastore is accessed directly by Hive (via JDBC).
-

HCatalog

- A layer built on Hive Metastore.
 - Helps other tools like **Pig**, **MapReduce**, and **Spark** easily read/write Hive data.
 - Makes data sharing between tools easy.
-

WebHCat

- A **REST API** for Hive and HCatalog.
 - Lets you run Hive, Pig, or Hadoop jobs using **HTTP calls** (like a web service).
-

Processing and Resource Management

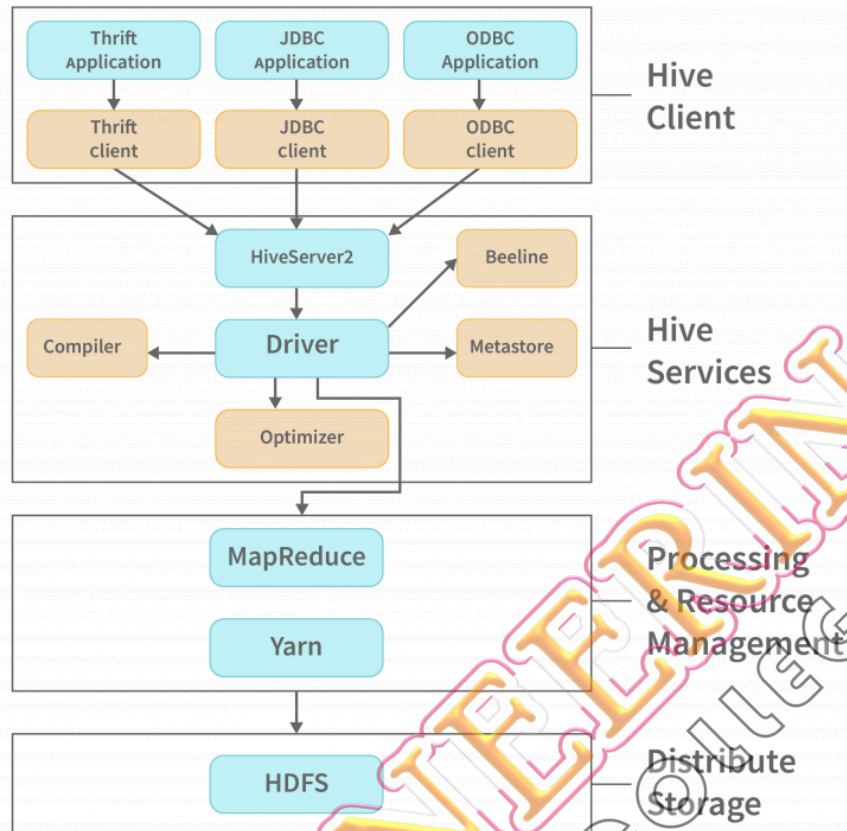
- Hive uses **MapReduce** (by default) to process queries.
 - It splits large tasks into smaller parts, runs them in parallel, and then combines the results.
-

Distributed Storage (HDFS)

Distributed Storage (HDFS)

- Hive stores data in **HDFS** (Hadoop Distributed File System).
- Data is stored across many machines so it can be processed quickly and reliably.

Hive Architecture & Its Components



InterviewBit

Q2 Compare and Contrast

- (i) Apache Pig vs Map-Reduce
- (ii) Pig vs SQL
- (iii) Pig vs HIVE?

(i) Apache Pig vs MapReduce

Feature	Apache Pig	MapReduce
Language	High-level (Pig Latin)	Low-level (Java/Python)
Ease of Use	Easier (fewer lines of code)	More complex (requires more coding)
Abstraction	Provides data flow abstraction	Requires manual implementation
Development Time	Faster development	Slower due to boilerplate code

Optimization	Automatic optimization (logical plans)	Manual optimization needed
Use Case	ETL, batch processing	Fine-grained control over processing
Schema Handling	Supports schema (optional)	No built-in schema support
UDF Support	Supports UDFs (Java, Python, etc.)	Native Java-based UDFs

Key Takeaway:
Pig simplifies Hadoop programming by abstracting MapReduce complexities, whereas MapReduce offers more control but requires more coding effort.

(ii) Pig vs SQL

Feature	Apache Pig	SQL
Language	Procedural (Pig Latin)	Declarative (SELECT, JOIN, etc.)
Execution Model	Data flow (step-by-step)	Set-based operations
Schema	Schema optional (flexible)	Schema required (rigid)
Use Case	ETL, complex transformations	Querying structured data
Optimization	Pig engine optimizes execution	Database engine optimizes queries
Data Types	Supports complex nested data	Primarily flat tables
Platform	Runs on Hadoop (batch processing)	Runs on RDBMS (real-time queries)

Key Takeaway:
Pig is better for ETL and batch processing on Hadoop, while SQL is ideal for structured querying in relational databases.

(iii) Pig vs Hive

Feature	Apache Pig	Hive
Language	Pig Latin (procedural)	HiveQL (SQL-like, declarative)
Use Case	ETL, data pipelines	Data warehousing, analytics
Schema	Optional (schema-on-read)	Enforced (schema-on-write)

Execution	Converts to MapReduce/Tez/Spark jobs	Converts to MapReduce/Tez/Spark jobs
Performance	Slightly slower (general-purpose)	Optimized for query performance
User Base	Developers, data engineers	Analysts, SQL users
Metadata	No metadata storage	Uses metastore (like RDBMS)

Q3. Explore various execution models of PIG..?

Apache Pig can run in different ways (called **execution models**) depending on where and how you want to process your data. Think of it like choosing between different vehicles for a trip:

1. MapReduce (Old, Reliable Truck)

- Default mode, runs on Hadoop.
- Processes data step-by-step, but slow because it writes to disk often.
- Good for basic batch jobs.

2. Tez (Faster Express Train)

- Processes data in memory, reducing delays.
- Best for complex jobs with multiple steps (like joins, aggregations).
- Faster than MapReduce but needs Tez installed.

3. Spark (Super-Fast Sports Car)

- Uses RAM for super-fast processing.
- Best for machine learning, real-time analytics, and big data.
- Needs Spark setup.

4. Local Mode (Bicycle for Testing)

- Runs on your own computer (no Hadoop needed).
- Great for testing small scripts before running on a big cluster.
- Not for real big data.

Which One Should You Use?

- Need speed? → Spark (if available) or Tez.
- Using old Hadoop? → MapReduce (default).
- Just testing? → Local Mode.

Q4. Applications on Big Data using Pig, Hive and HBase..?

Applications of Big Data Tools: Pig, Hive, and HBase

These tools serve different purposes in **Big Data processing**, from batch analytics to real-time queries. Below are their key applications:

1. Apache Pig (ETL & Data Pipelines)

Best for: Processing raw data into structured formats.

Applications:

✓ ETL (Extract, Transform, Load)

- Cleanse and transform unstructured/semi-structured data (logs, JSON, CSV).
- Example: Preparing social media data for analysis.

✓ Batch Processing

- Aggregations, filtering, and joins on large datasets.
- Example: Calculating monthly sales trends.

✓ Prototyping Algorithms

- Quickly test data processing logic before coding in Java/Python.

✓ Log Analysis

- Parse server logs to detect errors or user behavior.

Example Use Case:

A retail company uses Pig to clean and merge customer transaction logs before loading them into a data warehouse.

2. Apache Hive (Data Warehousing & Analytics)

Best for: SQL-like querying on Hadoop.

Applications:

✓ Data Warehousing

- Store and query structured data (like a Hadoop-based "database").
- Example: Analyzing years of sales records.

✓ Business Intelligence (BI) Reporting

- Connect with tools like Tableau/Power BI for dashboards.

✓ Ad-Hoc Queries

- Run SQL queries on big datasets (e.g., "Top 10 customers by revenue").

✓ Time-Series Analysis

- Analyze trends over time (daily, monthly metrics).

Example Use Case:

A bank uses Hive to generate quarterly financial reports from transaction data.

3. Apache HBase (Real-Time NoSQL Database)

Best for: Low-latency, random read/write access.

Applications:

✓ Real-Time Analytics

- Fraud detection, instant user recommendations.

✓ Time-Series Data

- IoT sensor data, stock market tickers.

✓ Transactional Systems

- E-commerce carts, messaging apps (WhatsApp uses HBase).

✓ Sparse Data Storage

- Efficiently stores data with many empty fields.

Example Use Case:

A telecom company uses HBase to store call records and instantly retrieve customer call history.